

A 29-AND Circuit for the AES S-box

Authors (unordered): {GPT-5.6 Pro, [umizame](#)}

12 July 2026

Abstract

The NIST Circuit Complexity project defines multiplicative complexity in a one-bit Boolean-circuit model and lists a forward AES S-box circuit with 32 AND gates. We give a completely explicit construction with 29 AND gates. The construction is specified by eight input-linear coordinates, 24 products of linear forms, five intervening products whose factors are given recursively, four post-middle coordinates, and eight affine output expressions. Nine of the 24 outer products are available before the five middle products and fifteen are available afterward, giving the legal nonlinear schedule

$$9 + 5 + 15 = 29.$$

The complete formulas and the complete 256-entry evaluation table are included. The latter agrees entry by entry with the forward S-box in FIPS 197. Hence the construction proves $\text{MC}(\text{AES-Sbox}) \leq 29$. A concrete straight-line implementation has 29 AND, 195 XOR, and 4 NOT instructions, ordinary gate depth 35, and AND-depth 6. Independent verifiers check the staged formulas, the FIPS algebraic definition and table, the implementation grammar and dependencies, a normalized XOR–AND graph, the derivation from the official 32-AND NIST program, and deterministic reconstruction. The result is an upper bound only; no minimality claim is made.

Contents

1	Problem statement and exact conventions	2
1.1	NIST circuit model and multiplicative complexity	2
1.2	The FIPS 197 function	2
1.3	Hexadecimal mask notation	3
2	The explicit staged 29-AND construction	3
2.1	Input-linear coordinates and the 24 outer products	3
2.2	Five recursively available middle products	3
2.3	Post-middle coordinates, late products, and outputs	4
3	Correctness and the multiplicative-complexity bound	4
4	Independent derivation from the official NIST circuit	5
5	An eight-product identity over $\mathbb{F}_4$	6
6	Formal quadratic bridge	7
7	Verification separation	8
8	Conclusion	8

A	Complete staged decomposition	9
A.1	Input-linear coordinates	9
A.2	Five retained middle products	10
A.3	Post-middle coordinates	10
A.4	Output reconstruction	10
A.5	Formal target and reconstruction masks	11
B	Complete finite evaluation and normalized graph	11
B.1	Normalized XOR–AND graph	12
C	Repository artifacts and standalone verification	13
C.1	Standalone exhaustive implementation verifier	13

1 Problem statement and exact conventions

1.1 NIST circuit model and multiplicative complexity

NIST defines the multiplicative size of a Boolean circuit as the number of its nonlinear gates and the multiplicative complexity of a Boolean or vectorial Boolean function as the minimum multiplicative size among circuits computing it. The NIST circuit collection uses the one-bit basis {AND, XOR, NOT} for this problem [1]. Over $\mathbb{F}_2$, XOR and NOT are affine and AND is nonlinear; therefore multiplicative size in this basis is exactly the number of AND gates.

For the forward AES S-box, NIST lists a circuit with 32 AND gates and 81 XOR/XNOR gates, 113 gates in total, ordinary depth 27, and AND-depth 6 [2]. The corresponding official straight-line program contains 32 AND, 77 XOR, and 4 XNOR instructions [3]. NIST asks whether fewer than 32 nonlinear gates suffice [1]. The theorem proved below supplies a 29-AND upper bound.

Definition 1 (Circuit model). A circuit in this paper has eight input bits, arbitrary fan-out, and one-bit gates of the forms

$$a \oplus b, \quad a \wedge b, \quad 1 \oplus a.$$

Every gate input must be either an external input or the output of an earlier gate. The multiplicative size is the number of $\wedge$ -gates. The multiplicative complexity $\text{MC}(F)$ is the minimum multiplicative size over all such circuits computing F .

1.2 The FIPS 197 function

Write an external byte as

$$x = (U_0 U_1 \cdots U_7)_2 = \sum_{j=0}^7 U_j 2^{7-j},$$

so U_0 is the most significant bit. For the polynomial representation, write instead $x = \sum_{i=0}^7 b_i 2^i$ and associate the polynomial $\sum_{i=0}^7 b_i X^i$. FIPS 197 defines the forward S-box by inversion in

$$\mathbb{F}_2[X]/(X^8 + X^4 + X^3 + X + 1), \tag{1}$$

with zero mapped to zero, followed by an affine map with constant byte 63 [4]. If y_i are the least-significant-bit-first coefficients of the inverse, then

$$s_i = y_i \oplus y_{i+4} \oplus y_{i+5} \oplus y_{i+6} \oplus y_{i+7} \oplus c_i, \quad c = 63, \tag{2}$$

where subscripts are reduced modulo 8. The external output notation is $\text{AES-Sbox}(x) = (S_0 S_1 \cdots S_7)_2$, with $S_0 = s_7$ and $S_7 = s_0$. FIPS 197 also prints all 256 output bytes in its S-box table. Both descriptions are checked independently in the verification described below.

1.3 Hexadecimal mask notation

Definition 2 (Affine masks). For an ordered tuple of bits $B = (b_0, \dots, b_{r-1})$ and an integer $m = \sum_{j=0}^{r-1} m_j 2^j$, define

$$\langle m \rangle_B = \bigoplus_{j:m_j=1} b_j. \quad (3)$$

Masks are printed in hexadecimal. Thus the least significant bit of the *mask* selects the first member of the declared tuple; this convention is independent of whether a byte bit is most or least significant.

For example, with $B_U = (U_0, \dots, U_7)$,

$$\langle 69 \rangle_{B_U} = U_0 \oplus U_3 \oplus U_5 \oplus U_6.$$

This convention makes every formula below unambiguous and directly machine-checkable.

2 The explicit staged 29-AND construction

This section defines a Boolean map $C_{29} : \mathbb{F}_2^8 \rightarrow \mathbb{F}_2^8$ without reference to any previous circuit. Every affine expression is specified by a mask, and every nonlinear operation is displayed as one product of two affine expressions.

2.1 Input-linear coordinates and the 24 outer products

Let

$$B_U = (U_0, U_1, \dots, U_7).$$

Define $L_0, \dots, L_3, R_0, \dots, R_3$ by the eight masks in [table 2](#). These are linear functions of the input and require no AND gate.

Eventually there will also be four bits $Z_0, \dots, Z_3$. Put

$$B_C = (L_0, L_1, L_2, L_3, R_0, R_1, R_2, R_3, Z_0, Z_1, Z_2, Z_3). \quad (4)$$

For each row (u_j, v_j) of [table 1](#), define

$$P_j = \langle u_j \rangle_{B_C} \wedge \langle v_j \rangle_{B_C}. \quad (5)$$

The nine products

$$E = (P_{01}, P_{02}, P_{03}, P_{09}, P_{10}, P_{11}, P_{17}, P_{18}, P_{19}) \quad (6)$$

have zero coefficients on all four Z -coordinates, as is visible directly from their factor masks. They can therefore be computed immediately after L and R . The other fifteen products are deferred until Z has been defined.

2.2 Five recursively available middle products

Use the padded ordered tuple

$$B_M = (1, U_0, \dots, U_7, E, M_1, M_2, M_3, M_4, M_5). \quad (7)$$

For $k = 1, \dots, 5$, the row for M_k in [table 3](#) contains two masks a_k, b_k . Define recursively

$$M_k = \langle a_k \rangle_{B_M} \wedge \langle b_k \rangle_{B_M}. \quad (8)$$

Every coefficient on $M_k, M_{k+1}, \dots, M_5$ is zero in that row. Consequently the right side of [equation \(8\)](#) uses only the constant, inputs, the nine already-computed products in E , and earlier middle values $M_1, \dots, M_{k-1}$. The recursion is therefore a legal straight-line schedule rather than a simultaneous or circular definition.

2.3 Post-middle coordinates, late products, and outputs

After M_5 has been computed, define $Z_0, \dots, Z_3$ by the four masks in [table 4](#), all interpreted in B_M . The full coordinate tuple B_C in [equation \(4\)](#) is then available, so the remaining fifteen rows of [table 1](#) define the late products

$$D = (P_{04}, P_{05}, P_{06}, P_{07}, P_{08}, P_{12}, P_{13}, P_{14}, P_{15}, P_{16}, P_{20}, P_{21}, P_{22}, P_{23}, P_{24}). \quad (9)$$

Finally set

$$B_O = (B_M, D)$$

and define $S_0, \dots, S_7$ by the eight masks in [table 5](#), interpreted in B_O .

Lemma 3 (Availability and nonlinear count). *The construction above is an acyclic circuit in the model of [definition 1](#). It contains exactly 29 AND gates, with schedule $9 + 5 + 15$.*

Proof. The eight L, R coordinates are linear in the inputs. By inspection of the nine masks listed in [equation \(6\)](#), their factors involve only L, R , so those nine products are available first. In the row defining M_k , all mask bits belonging to M_k and later middle values are zero; induction on k therefore proves that each of the five middle products is available when it is formed. The four Z -coordinates are affine in values then available. This makes every factor of the fifteen late products available. The outputs are affine combinations of already-computed values. No affine step contributes an AND gate. The only nonlinear operations are the nine early products, five middle products, and fifteen late products, totaling $9 + 5 + 15 = 29$. $\square$

3 Correctness and the multiplicative-complexity bound

The construction has only eight inputs, so functional correctness can be certified without sampling or an unproved algebraic identification: evaluate all $2^8 = 256$ inputs. [section B](#) gives the complete output table obtained from the recursion in [equations \(5\) and \(8\)](#). Row r and column c contain the output for input byte $\mathbf{rc}$. It is entrywise identical to FIPS 197, Table 4.

Lemma 4 (Finite-domain equality). *Let $F, G : \mathbb{F}_2^8 \rightarrow \mathbb{F}_2^8$. If $F(x) = G(x)$ for every one of the 256 input bytes, then $F = G$.*

Proof. The 256 bytes are exactly all elements of $\mathbb{F}_2^8$. Pointwise equality on this complete domain is the definition of equality of the two functions. $\square$

Theorem 5 (Constructive 29-AND upper bound). *For the forward AES S-box specified by FIPS 197, in the circuit model of [definition 1](#),*

$$\text{MC}(\text{AES-Sbox}) \leq 29.$$

Proof. By [lemma 3](#), C_{29} is a legal circuit with exactly 29 AND gates. Direct recursive evaluation of its displayed formulas gives [table 7](#). The same table is the complete FIPS 197 forward S-box table. Therefore $C_{29}(x) = \text{AES-Sbox}(x)$ for every input byte; by [lemma 4](#), $C_{29} = \text{AES-Sbox}$. Since multiplicative complexity is the minimum AND count over all circuits computing the function, the existence of this circuit gives $\text{MC}(\text{AES-Sbox}) \leq 29$. $\square$

The table is not the only correctness check. The standalone verifier reproduced in [section C](#) first checks the literal 256-byte FIPS table against the field inversion and affine map in [equations \(1\) and \(2\)](#); it then parses and exhaustively evaluates the released straight-line implementation. A separate staged verifier interprets precisely the mask bases and recursions in this paper, enforces every availability restriction, and again compares all 256 values with FIPS. Thus the field definition, printed table, staged construction, and compiled straight-line program are pairwise connected by explicit finite checks.

Corollary 6 (Concrete straight-line implementation). *There is a straight-line implementation of C_{29} containing*

$$29 \text{ AND} + 195 \text{ XOR} + 4 \text{ NOT} = 228$$

instructions. Its ordinary gate depth is 35 and its AND-depth is 6.

Proof. The strict parser listed in [section C](#) accepts only the declared one-bit grammar, verifies that every operand is previously defined, prohibits wire redefinition, requires all declared temporaries and outputs, and counts the accepted instructions. It obtains the stated tally. Giving inputs depth zero and assigning each gate one plus the maximum operand depth yields output depth 35. For AND-depth, the same recurrence adds one only at an AND gate and obtains 6. Because the verified program order is acyclic, these recurrences equal the corresponding longest-path definitions. The same parser exhaustively verifies the implementation output against FIPS on all 256 inputs. $\square$

4 Independent derivation from the official NIST circuit

The proof of [theorem 5](#) is already complete and does not assume the correctness of another circuit. This section records how the construction was obtained and supplies a second, structurally different audit.

The official NIST 32-AND SLP numbers its nonlinear outputs in program order. The first nine are

$$t_{24}, t_{25}, t_{27}, t_{29}, t_{30}, t_{32}, t_{34}, t_{35}, t_{37},$$

the next five are

$$t_{48}, t_{50}, t_{54}, t_{58}, t_{61},$$

and the last eighteen are $t_{68}, t_{69}, \dots, t_{85}$. Hence its nonlinear schedule is $9 + 5 + 18$. The SLP states that it is based on the low-multiplicative-complexity AES construction of Boyar, Matthews, and Peralta, with an affine-layer improvement [3, 5].

The following coordinate vectors are exact named-wire identities in that SLP:

$$\begin{aligned} L &= (t_8, t_{11}, t_2, t_{10}), \\ R &= (t_{13}, t_{15}, t_{21}, t_6), \\ Z &= (t_{66}, t_{59}, t_{65}, t_{62}). \end{aligned} \tag{10}$$

The first two are available before any AND gate and equal the input-linear forms printed in [table 2](#). The vector Z is available after the five middle AND gates. The new construction retains those five middle functions, replaces the 27 outer products by the 24 products of [table 1](#), and supplies explicit affine reconstruction masks for every retained factor, every Z_i , and every output. The exact masks, rather than an assertion of span membership, are printed in [tables 3 to 5](#).

For an additional symbolic audit, every NIST wire is processed as an affine expression over the constant, the eight inputs, and 32 independent symbols representing the successive NIST AND outputs. XOR, XNOR, and NOT update this expression exactly; encountering an AND introduces the next independent symbol. This syntactic calculation does not assume a truth-table identity between unrelated wires. It proves the coordinate identities in [equation \(10\)](#), the factor-coordinate forms of all 27 outer AND gates, and the exact coefficient vectors by which outer AND outputs enter the ten middle factors and eight final outputs.

The remaining part of that symbolic bridge is described in [section 6](#). Its role is provenance and redundancy: the direct staged formulas and complete FIPS evaluation remain sufficient for the theorem even if the NIST derivation is ignored.

5 An eight-product identity over $\mathbb{F}_4$

The 24 outer products have a compact algebraic organization. Let

$$K = \mathbb{F}_4 = \mathbb{F}_2[\omega]/(\omega^2 + \omega + 1)$$

and write $x = x_0\omega + x_1$, $y = y_0\omega + y_1$, and $z = z_0\omega + z_1$. Define

$$\begin{aligned} p_1 &= x_0y_0, & p_2 &= x_1y_1, & p_3 &= (x_0 \oplus x_1)(y_0 \oplus y_1), & (11) \\ p_4 &= (x_0 \oplus y_0 \oplus y_1 \oplus z_1)(x_1 \oplus y_0 \oplus z_0), \\ p_5 &= (x_1 \oplus y_0 \oplus y_1 \oplus z_1)(x_0 \oplus x_1 \oplus y_0 \oplus z_0), \\ p_6 &= (x_1 \oplus y_0 \oplus y_1 \oplus z_0 \oplus z_1)(x_0 \oplus y_1 \oplus z_0), \\ p_7 &= (x_1 \oplus y_0 \oplus z_1)(x_0 \oplus x_1 \oplus y_1 \oplus z_0), \\ p_8 &= (x_1 \oplus y_0 \oplus z_0 \oplus z_1)(x_0 \oplus y_0 \oplus y_1 \oplus z_0). & (12) \end{aligned}$$

Then define six affine reconstructions by

$$(xy)_0 = p_2 \oplus p_3, \quad (xy)_1 = p_1 \oplus p_2, \quad (13)$$

$$\begin{aligned} (yz)_0 &= y_0 \oplus y_1 \oplus p_1 \oplus p_2 \oplus p_3 \oplus p_6 \oplus p_8, \\ (yz)_1 &= y_1 \oplus p_5 \oplus p_6 \oplus p_7 \oplus p_8, & (14) \end{aligned}$$

$$\begin{aligned} (zx)_0 &= x_1 \oplus p_1 \oplus p_2 \oplus p_3 \oplus p_4 \oplus p_5, \\ (zx)_1 &= x_1 \oplus y_1 \oplus z_0 \oplus p_1 \oplus p_2 \oplus p_3 \oplus p_6 \oplus p_7. & (15) \end{aligned}$$

Here $(ab)_{0\omega} + (ab)_1$ denotes the displayed pair of reconstructed bits.

Proposition 7 (Filtered $\mathbb{F}_4$ triangle identity). *For every $(x, y, z) \in K^3$, the quantities in equations (13) to (15) are exactly the coordinates of xy, yz, zx . Products p_1, p_2, p_3 use only x, y , while $p_4, \dots, p_8$ may use z .*

Proof. Since $\omega^2 = \omega + 1$,

$$(a_0\omega + a_1)(b_0\omega + b_1) = (a_0b_0 \oplus a_0b_1 \oplus a_1b_0)\omega \oplus (a_0b_0 \oplus a_1b_1). \quad (16)$$

Expand the right sides of equations (13) to (15), substitute equations (11) and (12), and cancel repeated monomials over $\mathbb{F}_2$. The six resulting algebraic normal forms are

$$\begin{aligned} (xy)_0 &= x_0y_0 \oplus x_0y_1 \oplus x_1y_0, & (xy)_1 &= x_0y_0 \oplus x_1y_1, \\ (yz)_0 &= y_0z_0 \oplus y_0z_1 \oplus y_1z_0, & (yz)_1 &= y_0z_0 \oplus y_1z_1, \\ (zx)_0 &= z_0x_0 \oplus z_0x_1 \oplus z_1x_0, & (zx)_1 &= z_0x_0 \oplus z_1x_1. \end{aligned}$$

These are precisely the two coordinates in equation (16). The availability statement follows directly from the eight factor formulas. $\square$

To obtain the 24 rows in table 1, first apply the invertible binary transform

$$\Phi(a_0, a_1, a_2, a_3) = ((a_2 \oplus a_3)\omega + a_3, a_1\omega + a_0). \quad (17)$$

Its four binary row masks are **c, 8, 2, 1**; the compact tag **128c** records them from high to low hexadecimal digit. Invertibility is immediate from

$$a_3 = q_1, \quad a_2 = q_0 \oplus q_1, \quad a_1 = q_2, \quad a_0 = q_3,$$

where $(q_0, q_1, q_2, q_3) = (a_2 \oplus a_3, a_3, a_1, a_0)$.

Apply [proposition 7](#) three times: to the first components of $\Phi(L), \Phi(R), \Phi(Z)$, to their second components, and to the componentwise sums of those two triples. This produces three blocks of eight products. In each block, three products are early and five are late, so the total filtration is nine early and fifteen late. This observation explains the factor pattern in [table 1](#). It is not an assumption that a particular field structure on four-bit vectors matches the NIST circuit: the exact relation to the required NIST functions is checked by the explicit reconstruction masks and the formal bridge below.

6 Formal quadratic bridge

The following calculation independently checks why the 24 factor pairs can replace the outer portion of the 32-AND NIST construction.

Let $V = \mathbb{F}_2^{12}$ with ordered coordinates B_C from [equation \(4\)](#). For $u \in V$, let $\ell_u = \langle u \rangle_{B_C}$, now treating the twelve coordinates as independent Boolean indeterminates.

Lemma 8 (Product signature and diagonal correction). *In the Boolean polynomial ring with $X_i^2 = X_i$,*

$$\ell_u(X)\ell_v(X) = \bigoplus_{i < j} (u_i v_j \oplus u_j v_i) X_i X_j \oplus \bigoplus_i u_i v_i X_i. \quad (18)$$

Thus the off-diagonal quadratic signature is the decomposable two-form $u \wedge v \in \Lambda^2 V$, while the diagonal linear correction has mask $u \& v$.

Proof. Expand the product. For $i < j$, the coefficient of $X_i X_j$ is the XOR of the two ordered contributions $u_i v_j$ and $u_j v_i$. The diagonal contribution is $u_i v_i X_i^2 = u_i v_i X_i$. Grouping the terms gives [equation \(18\)](#). $\square$

Encode $u \wedge v$ in the lexicographically ordered pair basis

$$(0, 1), (0, 2), \dots, (0, 11), (1, 2), \dots, (10, 11).$$

A symbolic pass through the NIST SLP gives 27 linearly independent signatures for its nine early and 18 late outer AND outputs. Projecting the ten middle-factor expressions and eight output expressions onto those 27 outer symbols, and then mapping through the corresponding signatures, gives an ordered four-dimensional pre-middle space Q_{pre} and an ordered twelve-dimensional total space Q , with

$$Q_{\text{pre}} \subset Q \subset \Lambda^2 V, \quad \dim Q_{\text{pre}} = 4, \quad \dim Q = 12.$$

The twelve target signatures and the 24-bit product-selection masks are printed in [table 6](#); the first four selection masks use only the nine early products.

Proposition 9 (Exact formal reconstruction). *For every row of [table 6](#), the selected products from [table 1](#), together with the diagonal correction prescribed by [lemma 8](#), equal the indicated quadratic polynomial on all $2^{12} = 4096$ assignments of the formal coordinates. The first four identities use no late product.*

Proof. For a selection set J , compute

$$d = \bigoplus_{j \in J} (u_j \wedge v_j), \quad r = \bigoplus_{j \in J} (u_j \& v_j).$$

The stored selection mask is checked to give exactly the target signature d . Applying [lemma 8](#) term by term then gives the polynomial identity

$$q(X) = \bigoplus_{j \in J} \ell_{u_j}(X) \ell_{v_j}(X) \oplus \ell_r(X).$$

This coefficient argument proves the identity. As a redundant check of coordinate and bit ordering, both sides are also evaluated on all 4096 assignments. Inspection of the first four selection masks shows that their set bits occur only at the nine early product positions. $\square$

The formal calculation concerns the quadratic outer contribution. The full affine corrections, constant terms, recursive middle dependencies, post-middle coordinates, and output expressions are not left implicit: they are the explicit staged masks in [tables 3 to 5](#). Their exhaustive eight-input evaluation is the direct correctness proof used in [theorem 5](#).

7 Verification separation

The result is checked through independent representations so that a defect in one representation is unlikely to be silently inherited by all others.

1. The staged-certificate checker parses the 24 factor pairs and all affine masks, enforces the nine-before-five-before-fifteen availability filtration, and evaluates all 256 inputs against a literal FIPS table.
2. The direct straight-line checker strictly parses declarations, delimiters, gate arities, dependencies, and wire names; it recomputes the exact tally and both depths, independently derives the FIPS S-box by finite-field arithmetic, checks that derivation against the literal table, and evaluates the circuit on all inputs.
3. A bit-parallel evaluator represents every wire by its complete 256-bit truth table. A separate C99 parser and evaluator repeats the finite circuit test.
4. Both straight-line implementations are normalized by absorbing every affine operation into masks over the constant, inputs, and preceding AND outputs. Their 29 ordered factor pairs and eight output masks agree exactly with the printed XOR-AND graph certificate.
5. The NIST bridge derives its target from the official 32-AND SLP rather than accepting a target from the 24-product witness; it verifies the formal identities on all 4096 coordinate assignments and the actual staged realization on all 256 AES inputs.
6. A clean-tree rebuild regenerates the readable circuit, resynthesizes only its affine network, regenerates the normalized graph, and compares all generated artifacts byte for byte.

These checks have deliberately different proof obligations. The first two alone each give a complete finite correctness check for the upper bound. The remaining checks establish provenance, representation consistency, and deterministic reproducibility. None of them is used to assert a 29-gate lower bound.

8 Conclusion

The explicit staged construction in this paper is a legal one-bit Boolean circuit with nonlinear schedule $9 + 5 + 15 = 29$. Its complete 256-entry output table is the FIPS 197 forward AES S-box table. Therefore $\text{MC}(\text{AES-Sbox}) \leq 29$. A concrete implementation uses 228 one-bit instructions and has AND-depth 6. The derivation from the official NIST 32-AND construction, the three-block $\mathbb{F}_4$ identity, the formal quadratic bridge, and multiple independent evaluators provide additional checks, but the theorem ultimately rests on the explicit formulas and complete finite evaluation. The construction establishes an upper bound and does not establish optimality.

A Complete staged decomposition

All masks in this appendix use [definition 2](#). The product table supplies every factor of all 24 outer AND gates. The staged tables then supply every input coordinate, every factor of the five middle AND gates, every post-middle coordinate, and every output expression. Together these tables are a complete specification of C_{29} .

Table 1: Complete 24-product certificate. A factor mask uses coordinate order $(L_0, \dots, L_3, R_0, \dots, R_3, Z_0, \dots, Z_3)$. Multiplication is commutative, so the displayed factor order has no mathematical significance.

product	stage	block	first factor	second factor
P_{01}	early	0	$\ell_{0c0} = R_2 \oplus R_3$	$\ell_{00c} = L_2 \oplus L_3$
P_{02}	early	0	$\ell_{080} = R_3$	$\ell_{008} = L_3$
P_{03}	early	0	$\ell_{040} = R_2$	$\ell_{004} = L_2$
P_{04}	late	0	$\ell_{84c} = L_2 \oplus L_3 \oplus R_2 \oplus Z_3$	$\ell_{cc8} = L_3 \oplus R_2 \oplus R_3 \oplus Z_2 \oplus Z_3$
P_{05}	late	0	$\ell_{848} = L_3 \oplus R_2 \oplus Z_3$	$\ell_{cc4} = L_2 \oplus R_2 \oplus R_3 \oplus Z_2 \oplus Z_3$
P_{06}	late	0	$\ell_{448} = L_3 \oplus R_2 \oplus Z_2$	$\ell_{c8c} = L_2 \oplus L_3 \oplus R_3 \oplus Z_2 \oplus Z_3$
P_{07}	late	0	$\ell_{8c8} = L_3 \oplus R_2 \oplus R_3 \oplus Z_3$	$\ell_{c84} = L_2 \oplus R_3 \oplus Z_2 \oplus Z_3$
P_{08}	late	0	$\ell_{4c8} = L_3 \oplus R_2 \oplus R_3 \oplus Z_2$	$\ell_{c4c} = L_2 \oplus L_3 \oplus R_2 \oplus Z_2 \oplus Z_3$
P_{09}	early	1	$\ell_{020} = R_1$	$\ell_{002} = L_1$
P_{10}	early	1	$\ell_{010} = R_0$	$\ell_{001} = L_0$
P_{11}	early	1	$\ell_{030} = R_0 \oplus R_1$	$\ell_{003} = L_0 \oplus L_1$
P_{12}	late	1	$\ell_{132} = L_1 \oplus R_0 \oplus R_1 \oplus Z_0$	$\ell_{221} = L_0 \oplus R_1 \oplus Z_1$
P_{13}	late	1	$\ell_{131} = L_0 \oplus R_0 \oplus R_1 \oplus Z_0$	$\ell_{223} = L_0 \oplus L_1 \oplus R_1 \oplus Z_1$
P_{14}	late	1	$\ell_{331} = L_0 \oplus R_0 \oplus R_1 \oplus Z_0 \oplus Z_1$	$\ell_{212} = L_1 \oplus R_0 \oplus Z_1$
P_{15}	late	1	$\ell_{121} = L_0 \oplus R_1 \oplus Z_0$	$\ell_{213} = L_0 \oplus L_1 \oplus R_0 \oplus Z_1$
P_{16}	late	1	$\ell_{321} = L_0 \oplus R_1 \oplus Z_0 \oplus Z_1$	$\ell_{232} = L_1 \oplus R_0 \oplus R_1 \oplus Z_1$
P_{17}	early	$0 \oplus 1$	$\ell_{0e0} = R_1 \oplus R_2 \oplus R_3$	$\ell_{00e} = L_1 \oplus L_2 \oplus L_3$
P_{18}	early	$0 \oplus 1$	$\ell_{090} = R_0 \oplus R_3$	$\ell_{009} = L_0 \oplus L_3$
P_{19}	early	$0 \oplus 1$	$\ell_{070} = R_0 \oplus R_1 \oplus R_2$	$\ell_{007} = L_0 \oplus L_1 \oplus L_2$
P_{20}	late	$0 \oplus 1$	$\ell_{97e} = L_1 \oplus L_2 \oplus L_3 \oplus R_0 \oplus R_1 \oplus R_2 \oplus Z_0 \oplus Z_3$	$\ell_{ee9} = L_0 \oplus L_3 \oplus R_1 \oplus R_2 \oplus R_3 \oplus Z_1 \oplus Z_2 \oplus Z_3$
P_{21}	late	$0 \oplus 1$	$\ell_{979} = L_0 \oplus L_3 \oplus R_0 \oplus R_1 \oplus R_2 \oplus Z_0 \oplus Z_3$	$\ell_{ee7} = L_0 \oplus L_1 \oplus L_2 \oplus R_1 \oplus R_2 \oplus R_3 \oplus Z_1 \oplus Z_2 \oplus Z_3$
P_{22}	late	$0 \oplus 1$	$\ell_{779} = L_0 \oplus L_3 \oplus R_0 \oplus R_1 \oplus R_2 \oplus Z_0 \oplus Z_1 \oplus Z_2$	$\ell_{e9e} = L_1 \oplus L_2 \oplus L_3 \oplus R_0 \oplus R_3 \oplus Z_1 \oplus Z_2 \oplus Z_3$
P_{23}	late	$0 \oplus 1$	$\ell_{9e9} = L_0 \oplus L_3 \oplus R_1 \oplus R_2 \oplus R_3 \oplus Z_0 \oplus Z_3$	$\ell_{e97} = L_0 \oplus L_1 \oplus L_2 \oplus R_0 \oplus R_3 \oplus Z_1 \oplus Z_2 \oplus Z_3$
P_{24}	late	$0 \oplus 1$	$\ell_{7e9} = L_0 \oplus L_3 \oplus R_1 \oplus R_2 \oplus R_3 \oplus Z_0 \oplus Z_1 \oplus Z_2$	$\ell_{e7e} = L_1 \oplus L_2 \oplus L_3 \oplus R_0 \oplus R_1 \oplus R_2 \oplus Z_1 \oplus Z_2 \oplus Z_3$

A.1 Input-linear coordinates

The input bits are ordered $(U_0, \dots, U_7)$, with U_0 the most significant bit.

Table 2: The eight pre-middle coordinates as linear forms in $(U_0, \dots, U_7)$.

coordinate	mask	linear form
L_0	69	$U_0 \oplus U_3 \oplus U_5 \oplus U_6$
L_1	e7	$U_0 \oplus U_1 \oplus U_2 \oplus U_5 \oplus U_6 \oplus U_7$

coordinate	mask	linear form
L_2	41	$U_0 \oplus U_6$
L_3	c6	$U_1 \oplus U_2 \oplus U_6 \oplus U_7$
R_0	59	$U_0 \oplus U_3 \oplus U_4 \oplus U_6$
R_1	d9	$U_0 \oplus U_3 \oplus U_4 \oplus U_6 \oplus U_7$
R_2	74	$U_2 \oplus U_4 \oplus U_5 \oplus U_6$
R_3	86	$U_1 \oplus U_2 \oplus U_7$

A.2 Five retained middle products

For this table the mask basis is

$$(1, U_0, \dots, U_7, P_{01}, P_{02}, P_{03}, P_{09}, P_{10}, P_{11}, P_{17}, P_{18}, P_{19}, M_1, \dots, M_5).$$

A row defining M_k has zero coefficients on $M_k, \dots, M_5$; hence both factors are available before M_k is formed.

Table 3: Exact affine factors of the five middle AND gates.

gate	first factor	second factor
M_1	$\ell_{1\text{ecf6}} = U_0 \oplus U_1 \oplus U_3 \oplus U_4 \oplus U_5 \oplus U_6 \oplus P_{02} \oplus P_{03} \oplus P_{10} \oplus P_{11} \oplus P_{17} \oplus P_{18}$	$\ell_{1\text{d06a}} = U_0 \oplus U_2 \oplus U_4 \oplus U_5 \oplus P_{09} \oplus P_{11} \oplus P_{17} \oplus P_{18}$
M_2	$\ell_{2\text{b60a}} = U_0 \oplus U_2 \oplus P_{01} \oplus P_{02} \oplus P_{09} \oplus P_{10} \oplus P_{17} \oplus P_{19}$	$\ell_{730\text{ea}} = U_0 \oplus U_2 \oplus U_4 \oplus U_5 \oplus U_6 \oplus P_{09} \oplus P_{10} \oplus P_{18} \oplus P_{19} \oplus M_1$
M_3	$\ell_{75\text{afc}} = U_1 \oplus U_2 \oplus U_3 \oplus U_4 \oplus U_5 \oplus U_6 \oplus P_{01} \oplus P_{03} \oplus P_{09} \oplus P_{11} \oplus P_{18} \oplus P_{19} \oplus M_1$	$\ell_{2\text{e080}} = U_6 \oplus P_{10} \oplus P_{11} \oplus P_{17} \oplus P_{19}$
M_4	$\ell_{330\text{ea}} = U_0 \oplus U_2 \oplus U_4 \oplus U_5 \oplus U_6 \oplus P_{09} \oplus P_{10} \oplus P_{18} \oplus P_{19}$	$\ell_{140000} = M_1 \oplus M_3$
M_5	$\ell_{\text{b5afc}} = U_1 \oplus U_2 \oplus U_3 \oplus U_4 \oplus U_5 \oplus U_6 \oplus P_{01} \oplus P_{03} \oplus P_{09} \oplus P_{11} \oplus P_{18} \oplus P_{19} \oplus M_2$	$\ell_{2730\text{ea}} = U_0 \oplus U_2 \oplus U_4 \oplus U_5 \oplus U_6 \oplus P_{09} \oplus P_{10} \oplus P_{18} \oplus P_{19} \oplus M_1 \oplus M_4$

A.3 Post-middle coordinates

Table 4: The four post-middle coordinates in the same 23-element affine basis.

coordinate	mask	affine expression
Z_0	21d06a	$U_0 \oplus U_2 \oplus U_4 \oplus U_5 \oplus P_{09} \oplus P_{11} \oplus P_{17} \oplus P_{18} \oplus M_4$
Z_1	32e080	$U_6 \oplus P_{10} \oplus P_{11} \oplus P_{17} \oplus P_{19} \oplus M_3 \oplus M_4$
Z_2	49ecf6	$U_0 \oplus U_1 \oplus U_3 \oplus U_4 \oplus U_5 \oplus U_6 \oplus P_{02} \oplus P_{03} \oplus P_{10} \oplus P_{11} \oplus P_{17} \oplus P_{18} \oplus M_2 \oplus M_5$
Z_3	42b60a	$U_0 \oplus U_2 \oplus P_{01} \oplus P_{02} \oplus P_{09} \oplus P_{10} \oplus P_{17} \oplus P_{19} \oplus M_5$

A.4 Output reconstruction

For the output rows, append the late products in the order

$$(P_{04}, P_{05}, P_{06}, P_{07}, P_{08}, P_{12}, P_{13}, P_{14}, P_{15}, P_{16}, P_{20}, P_{21}, P_{22}, P_{23}, P_{24})$$

to the 23-element middle basis.

Table 5: Exact affine reconstruction of the eight output bits.

output	mask	affine expression
S_0	2491ba35ce	$U_0 \oplus U_1 \oplus U_2 \oplus U_5 \oplus U_6 \oplus U_7 \oplus P_{02} \oplus P_{09} \oplus P_{10} \oplus P_{19} \oplus M_2 \oplus M_3 \oplus M_4 \oplus P_{04} \oplus P_{05} \oplus P_{12} \oplus P_{15} \oplus P_{21} \oplus P_{24}$
S_1	257bba3b95	$1 \oplus U_1 \oplus U_3 \oplus U_6 \oplus U_7 \oplus P_{01} \oplus P_{03} \oplus P_{09} \oplus P_{10} \oplus P_{19} \oplus M_2 \oplus M_3 \oplus M_4 \oplus P_{04} \oplus P_{05} \oplus P_{06} \oplus P_{08} \oplus P_{12} \oplus P_{13} \oplus P_{14} \oplus P_{16} \oplus P_{21} \oplus P_{24}$
S_2	2e6532ee01	$1 \oplus P_{01} \oplus P_{02} \oplus P_{03} \oplus P_{10} \oplus P_{11} \oplus P_{17} \oplus P_{19} \oplus M_3 \oplus M_4 \oplus P_{05} \oplus P_{07} \oplus P_{13} \oplus P_{14} \oplus P_{20} \oplus P_{21} \oplus P_{22} \oplus P_{24}$
S_3	19dbba4b98	$U_2 \oplus U_3 \oplus U_6 \oplus U_7 \oplus P_{01} \oplus P_{03} \oplus P_{11} \oplus P_{19} \oplus M_2 \oplus M_3 \oplus M_4 \oplus P_{04} \oplus P_{05} \oplus P_{06} \oplus P_{08} \oplus P_{12} \oplus P_{14} \oplus P_{15} \oplus P_{16} \oplus P_{22} \oplus P_{23}$
S_4	189eba357c	$U_1 \oplus U_2 \oplus U_3 \oplus U_4 \oplus U_5 \oplus U_7 \oplus P_{02} \oplus P_{09} \oplus P_{10} \oplus P_{19} \oplus M_2 \oplus M_3 \oplus M_4 \oplus P_{04} \oplus P_{06} \oplus P_{07} \oplus P_{08} \oplus P_{12} \oplus P_{15} \oplus P_{22} \oplus P_{23}$
S_5	0c0332e152	$U_0 \oplus U_3 \oplus U_5 \oplus U_7 \oplus P_{10} \oplus P_{11} \oplus P_{17} \oplus P_{19} \oplus M_3 \oplus M_4 \oplus P_{05} \oplus P_{06} \oplus P_{21} \oplus P_{22}$
S_6	0db00b2bf9	$1 \oplus U_2 \oplus U_3 \oplus U_4 \oplus U_5 \oplus U_6 \oplus U_7 \oplus P_{01} \oplus P_{03} \oplus P_{10} \oplus P_{18} \oplus P_{19} \oplus M_2 \oplus P_{12} \oplus P_{13} \oplus P_{15} \oplus P_{16} \oplus P_{21} \oplus P_{22}$
S_7	19b508a44b	$1 \oplus U_0 \oplus U_2 \oplus U_5 \oplus P_{02} \oplus P_{10} \oplus P_{17} \oplus M_2 \oplus P_{05} \oplus P_{07} \oplus P_{12} \oplus P_{13} \oplus P_{15} \oplus P_{16} \oplus P_{22} \oplus P_{23}$

A.5 Formal target and reconstruction masks

Table 6: Formal quadratic reconstruction certificate derived from the NIST SLP. The first four rows form the pre-middle subspace.

row	first independent requirement	target signature	selected products
01 (Q_{pre})	first factor of M_1	000000003c281a058	030606
02 (Q_{pre})	second factor of M_1	000000001c381e050	030500
03 (Q_{pre})	first factor of M_2	00000000281416030	050303
04 (Q_{pre})	second factor of M_2	00000000241c0a078	060300
05 (Q)	output S_0	007e7943c281a0580	97481f
06 (Q)	output S_1	00fa6963c281a0580	97b8b8
07 (Q)	output S_2	005f31a241c0a0780	b83057
08 (Q)	output S_3	00841023c281a0580	67efb8
09 (Q)	output S_4	004c0863c281a0580	6748ef
10 (Q)	output S_5	002148e0c081c0480	300030
11 (Q)	output S_6	00e950a1c381e0500	30df00
12 (Q)	output S_7	00c81841c381e0500	67df57

B Complete finite evaluation and normalized graph

Table 7: Complete evaluation of the staged 29-AND certificate. Row and column headers are hexadecimal input nibbles; each body entry is the hexadecimal output byte. This table is entrywise identical to FIPS 197, Table 4.

	0	1	2	3	4	5	6	7	8	9	a	b	c	d	e	f
0	63	7c	77	7b	f2	6b	6f	c5	30	01	67	2b	fe	d7	ab	76
1	ca	82	c9	7d	fa	59	47	f0	ad	d4	a2	af	9c	a4	72	c0

	0	1	2	3	4	5	6	7	8	9	a	b	c	d	e	f
2	b7	fd	93	26	36	3f	f7	cc	34	a5	e5	f1	71	d8	31	15
3	04	c7	23	c3	18	96	05	9a	07	12	80	e2	eb	27	b2	75
4	09	83	2c	1a	1b	6e	5a	a0	52	3b	d6	b3	29	e3	2f	84
5	53	d1	00	ed	20	fc	b1	5b	6a	cb	be	39	4a	4c	58	cf
6	d0	ef	aa	fb	43	4d	33	85	45	f9	02	7f	50	3c	9f	a8
7	51	a3	40	8f	92	9d	38	f5	bc	b6	da	21	10	ff	f3	d2
8	cd	0c	13	ec	5f	97	44	17	c4	a7	7e	3d	64	5d	19	73
9	60	81	4f	dc	22	2a	90	88	46	ee	b8	14	de	5e	0b	db
a	e0	32	3a	0a	49	06	24	5c	c2	d3	ac	62	91	95	e4	79
b	e7	c8	37	6d	8d	d5	4e	a9	6c	56	f4	ea	65	7a	ae	08
c	ba	78	25	2e	1c	a6	b4	c6	e8	dd	74	1f	4b	bd	8b	8a
d	70	3e	b5	66	48	03	f6	0e	61	35	57	b9	86	c1	1d	9e
e	e1	f8	98	11	69	d9	8e	94	9b	1e	87	e9	ce	55	28	df
f	8c	a1	89	0d	bf	e6	42	68	41	99	2d	0f	b0	54	bb	16

B.1 Normalized XOR–AND graph

The following certificate absorbs all affine gates into masks. Mask bit 0 is the constant 1, mask bits 1–8 are $U_0, \dots, U_7$, and each later basis bit is the output of one preceding AND row. It therefore gives a compact, implementation-independent description of the 29 nonlinear gates and the eight final affine output masks.

Listing 1: Normalized 29-AND XAG certificate.

```

AES29-XAG 1
AND_COUNT 29
AND A01 1e4 10e
AND A02 10c 18c
AND A03 e8 82
AND A04 1b2 1ce
AND A05 b2 d2
AND A06 100 11c
AND A07 56 c0
AND A08 1be 15e
AND A09 1e8 19e
AND A10 1ecf6 1d06a
AND A11 2b60a 730ea
AND A12 75afc 2e080
AND A13 330ea 140000
AND A14 b5afc 2730ea
AND A15 42b7ec b5a94
AND A16 42b76e b5b9a
AND A17 49ed92 b5afe
AND A18 42b662 b5b72
AND A19 49ec9e b5b1a
AND A20 21d0a4 32e1e0
AND A21 21d1b8 32e02e
AND A22 133138 32e1fc
AND A23 21d10a 32e12e
AND A24 13318a 32e04e
AND A25 636748 39bb74
AND A26 6366d6 39bbb4
AND A27 5adcaa 39bb02
AND A28 636768 39ba5c
AND A29 5add14 39bb54
OUTPUT_COUNT 8
OUTPUT S0 2491ba35ce
OUTPUT S1 257bba3b95
OUTPUT S2 2e6532ee01
OUTPUT S3 19dbba4b98
OUTPUT S4 189eba357c
OUTPUT S5 c0332e152
OUTPUT S6 db00b2bf9
OUTPUT S7 19b508a44b
END

```

C Repository artifacts and standalone verification

This final appendix identifies the executable artifacts corresponding to the mathematical objects above. Internal repository paths are confined to this appendix.

Table 8: Principal released artifacts and fixed SHA-256 digests.

artifact	digest or role
circuits/aes-sbox-fwd-g228-a29-d35-ad6.slp	f41861c4b15bc78840708a5da2fab6ca9dd204c1e3e572cae49912c713bf18d3
circuits/aes-sbox-fwd-g455-a29-d35-ad6-transparent.slp	cecd0c7abba9920ff2af7ec204fcd01f97db135a4c43d8afd7f83d65c2e382ca
certificates/aes29.xag	862446d12293b21bfafbd7e958502301619a596e6bf3cb7a3b22f3f830926264
certificates/tower24_witness.txt	1e847a85e3abe783d98e3d4c48a7d9eb845caab96903c175657cce06119c6925
certificates/staged29_witness.txt	d72ca4cd9617b365587a3b826ab3a0d37489187cdd67fca18ff50b1d3d5e0083
baseline/aes-sbox-fwd-g113-a32-d27-ad6.slp	exact bytes of the official NIST SLP; 6ba4f63b832a1f4520a76cab5680e673fdd744ad2651918882eeecff7957698a
verify/verify_staged29.py	strict parser and exhaustive evaluator for the complete staged certificate
verify/minimal_verify.py	strict standalone SLP parser, FIPS algebraic/table cross-check, exhaustive evaluator, tally, and depth checker
verify/verify_tower24.py	independent NIST target derivation, formal 4096-point bridge, and staged 256-point check
verify/run_all.sh	redundant full verification suite

The shortest checks of the two direct certificates are

```
python3 verify/verify_staged29.py
python3 verify/minimal_verify.py
```

The complete redundant audit is

```
./verify/run_all.sh
```

The canonical paper source is `article/article.tex`. The release process first generates the included tables, compiles that TeX source to `article/article.pdf`, and only after the PDF exists renders the same expanded TeX source as the mathematical GitHub Pages document `docs/index.html`. The HTML is therefore a second rendering of the paper, not a separate informal article. Public copies of the circuit, certificates, verifier, and PDF are placed in `docs/files/` and checked byte for byte.

C.1 Standalone exhaustive implementation verifier

The following listing uses only the Python standard library. It contains a literal copy of the FIPS S-box table, independently computes the algebraic definition, strictly parses the complete released SLP container, checks exact counts and depths, and evaluates all 256 inputs.

Listing 2: Standalone exhaustive verifier.

```
#!/usr/bin/env python3
"""Standalone exhaustive verifier for the released 29-AND AES S-box SLP.

Only the Python standard library is used. The checker strictly parses the
complete circuit container, verifies all data dependencies and exact gate
counts, independently checks the literal FIPS 197 table against the algebraic
S-box definition, and evaluates the circuit on every one of the 256 inputs.
U0 and S0 are the most significant bits.
"""
from __future__ import annotations

from collections import Counter
from pathlib import Path
import hashlib
import re
import sys

ROOT = Path(__file__).resolve().parents[1]
DEFAULT = ROOT / "circuits" / "aes-sbox-fwd-g228-a29-d35-ad6.slp"
DEFAULT_SHA256 = "f41861c4b15bc78840708a5da2fab6ca9dd204c1e3e572cae49912c713bf18d3"
FIPS_SBOX = (
    0x63, 0x7c, 0x77, 0x7b, 0xf2, 0x6b, 0x6f, 0xc5, 0x30, 0x01, 0x67, 0x2b, 0xfe, 0xd7, 0xab, 0x76,
    0xca, 0x82, 0xc9, 0x7d, 0xfa, 0x59, 0x47, 0xf0, 0xad, 0xd4, 0xa2, 0xaf, 0x9c, 0xa4, 0x72, 0xc0,
    0xb7, 0xfd, 0x93, 0x26, 0x36, 0x3f, 0xf7, 0xcc, 0x34, 0xa5, 0xe5, 0xf1, 0x71, 0xd8, 0x31, 0x15,
    0x04, 0xc7, 0x23, 0xc3, 0x18, 0x96, 0x05, 0x9a, 0x07, 0x12, 0x80, 0xe2, 0xeb, 0x27, 0xb2, 0x75,
    0x09, 0x83, 0x2c, 0x1a, 0x1b, 0x6e, 0x5a, 0xa0, 0x52, 0x3b, 0xd6, 0xb3, 0x29, 0xe3, 0x2f, 0x84,
    0x53, 0xd1, 0x00, 0xed, 0x20, 0xfc, 0xb1, 0x5b, 0x6a, 0xcb, 0xbe, 0x39, 0x4a, 0x4c, 0x58, 0xcf,
    0xd0, 0xef, 0xaa, 0xfb, 0x43, 0x4d, 0x33, 0x85, 0x45, 0xf9, 0x02, 0x7f, 0x50, 0x3c, 0x9f, 0xa8,
    0x51, 0xa3, 0x40, 0x8f, 0x92, 0x9d, 0x38, 0xf5, 0xbc, 0xb6, 0xda, 0x21, 0x10, 0xff, 0xf3, 0xd2,
    0xcd, 0x0c, 0x13, 0xec, 0x5f, 0x97, 0x44, 0x17, 0xc4, 0xa7, 0x7e, 0x3d, 0x64, 0x5d, 0x19, 0x73,
    0x60, 0x81, 0x4f, 0xdc, 0x22, 0x2a, 0x90, 0x88, 0x46, 0xee, 0xb8, 0x14, 0xde, 0x5e, 0x0b, 0xdb,
    0xe0, 0x32, 0x3a, 0x0a, 0x49, 0x06, 0x24, 0x5c, 0xc2, 0xd3, 0xac, 0x62, 0x91, 0x95, 0xe4, 0x79,
    0xe7, 0xc8, 0x37, 0x6d, 0x8d, 0xd5, 0x4e, 0xa9, 0x6c, 0x56, 0xf4, 0xea, 0x65, 0x7a, 0xae, 0x08,
    0xba, 0x78, 0x25, 0x2e, 0x1c, 0xa6, 0xb4, 0xc6, 0xe8, 0xdd, 0x74, 0x1f, 0x4b, 0xbd, 0x8b, 0x8a,
    0x70, 0x3e, 0xb5, 0x66, 0x48, 0x03, 0xf6, 0x0e, 0x61, 0x35, 0x57, 0xb9, 0x86, 0xc1, 0x1d, 0x9e,
    0xe1, 0xf8, 0x98, 0x11, 0x69, 0xd9, 0x8e, 0x94, 0x9b, 0x1e, 0x87, 0xe9, 0xce, 0x55, 0x28, 0xdf,
    0x8c, 0xa1, 0x89, 0x0d, 0xbf, 0xe6, 0x42, 0x68, 0x41, 0x99, 0x2d, 0x0f, 0xb0, 0x54, 0xbb, 0x16,
)

def require(condition: bool, message: str) -> None:
    if not condition:
        raise SystemExit(f"FAIL: {message}")

def gf256_mul(a: int, b: int) -> int:
    result = 0
    for _ in range(8):
        if b & 1:
            result ^= a
        a = ((a << 1) & 0xFF) ^ (0x1B if a & 0x80 else 0)
        b >>= 1
    return result

def aes_sbox_algebraic(x: int) -> int:
    inverse = 0
    if x:
        inverse, base, exponent = 1, x, 254
        while exponent:
            if exponent & 1:
                inverse = gf256_mul(inverse, base)
            base = gf256_mul(base, base)
            exponent >>= 1
    output = 0
    for i in range(8):
        bit = (
            ((inverse >> i) & 1)
            ^ ((inverse >> ((i + 4) & 7)) & 1)
            ^ ((inverse >> ((i + 5) & 7)) & 1)
            ^ ((inverse >> ((i + 6) & 7)) & 1)
        )

```

```

        ^ ((inverse >> ((i + 7) & 7)) & 1)
        ^ ((0x63 >> i) & 1)
    )
    output |= bit << i
return output

def parse(path: Path) -> tuple[list[tuple[str, str, str, str | None]], tuple[str, ...], tuple[str, ...]]:

    operations: list[tuple[str, str, str, str | None]] = []
    inputs = tuple(f"U{i}" for i in range(8))
    outputs = tuple(f"S{i}" for i in range(8))
    state = "start"
    saw_inputs = saw_outputs = saw_internal = saw_syntax = False
    internal_count: int | None = None

    for lineno, raw in enumerate(path.read_text(encoding="utf-8").splitlines(), 1):
        line = raw.split("#", 1)[0].strip()
        if not line:
            continue

        if state == "start":
            require(re.fullmatch(r"begin circuit [A-Za-z0-9_-]+", line) is not None,
                    f"expected begin circuit at line {lineno}")
            state = "header"
            continue

        if state == "header":
            if line == "Inputs: U0:U7":
                require(not saw_inputs, f"duplicate input declaration at line {lineno}")
                saw_inputs = True
                continue
            if line == "Outputs: S0:S7":
                require(saw_inputs and not saw_outputs, f"misordered or duplicate output declaration at
line {lineno}")
                saw_outputs = True
                continue
            match = re.fullmatch(r"Internal: t1:t([1-9][0-9]*)", line)
            if match:
                require(saw_outputs and not saw_internal, f"misordered or duplicate internal declaration
at line {lineno}")
                internal_count = int(match.group(1))
                saw_internal = True
                continue
            if line == "GateSyntax: GateName Output Inputs":
                require(saw_internal and not saw_syntax, f"misordered or duplicate gate syntax at line {
lineno}")
                saw_syntax = True
                continue
            if line == "begin SLP":
                require(saw_inputs and saw_outputs and saw_internal and saw_syntax,
                    f"incomplete circuit header before line {lineno}")
                state = "slp"
                continue
            require(False, f"unexpected circuit-header line {lineno}: {line}")

        if state == "slp":
            if line == "end SLP":
                state = "after_slp"
                continue
            fields = line.split()
            if len(fields) == 4 and fields[0] in {"XOR", "AND"}:
                operations.append((fields[0], fields[1], fields[2], fields[3]))
            elif len(fields) == 3 and fields[0] == "NOT":
                operations.append((fields[0], fields[1], fields[2], None))
            else:
                require(False, f"illegal instruction at line {lineno}: {line}")
                continue

        if state == "after_slp":
            require(line == "end circuit", f"expected end circuit at line {lineno}")
            state = "done"

```

```

        continue

        require(False, f"trailing non-comment data at line {lineno}: {line}")

    require(state == "done", "missing or unbalanced circuit/SLP delimiters")
    require(internal_count is not None, "missing internal-wire declaration")
    require(bool(operations), "empty SLP")

    defined = set(inputs)
    temporary_outputs: set[str] = set()
    for kind, output, a, b in operations:
        require(output not in defined, f"wire redefinition: {output}")
        require(a in defined, f"use before definition: {a}")
        require(b is None or b in defined, f"use before definition: {b}")
        require(re.fullmatch(r"t[1-9][0-9]*|S[0-7]", output) is not None,
            f"undeclared output-wire name: {output}")
        if output.startswith("t"):
            number = int(output[1:])
            require(1 <= number <= internal_count, f"temporary outside declared range: {output}")
            temporary_outputs.add(output)
            defined.add(output)

    declared_temporaries = {f"t{i}" for i in range(1, internal_count + 1)}
    require(temporary_outputs == declared_temporaries, "declared and defined temporary-wire sets differ")
    require(all(name in defined for name in outputs), "missing output wire")
    return operations, inputs, outputs

def evaluate(operations: list[tuple[str, str, str, str | None]], inputs: tuple[str, ...], outputs: tuple[
    str, ...], x: int) -> int:
    wire = {name: (x >> (7 - i)) & 1 for i, name in enumerate(inputs)}
    for kind, output, a, b in operations:
        if kind == "XOR":
            wire[output] = wire[a] ^ wire[b] # type: ignore[index]
        elif kind == "AND":
            wire[output] = wire[a] & wire[b] # type: ignore[index]
        else:
            wire[output] = wire[a] ^ 1
    return sum(wire[name] << (7 - i) for i, name in enumerate(outputs))

def depths(operations: list[tuple[str, str, str, str | None]], inputs: tuple[str, ...], outputs: tuple[
    str, ...]) -> tuple[int, int]:
    gate_depth = {name: 0 for name in inputs}
    and_depth = dict(gate_depth)
    for kind, output, a, b in operations:
        operands = (a,) if b is None else (a, b)
        gate_depth[output] = 1 + max(gate_depth[name] for name in operands)
        and_depth[output] = max(and_depth[name] for name in operands) + (1 if kind == "AND" else 0)
    return max(gate_depth[name] for name in outputs), max(and_depth[name] for name in outputs)

def main() -> None:
    require(len(sys.argv) <= 2, "usage: minimal_verify.py [circuit.slp]")
    path = Path(sys.argv[1]) if len(sys.argv) == 2 else DEFAULT
    operations, inputs, outputs = parse(path)

    counts = Counter(kind for kind, _, _, _ in operations)
    expected_counts = Counter({"AND": 29, "XOR": 195, "NOT": 4})
    require(counts == expected_counts, f"unexpected operation tally: {dict(counts)}")
    require(len(operations) == 228, f"expected exactly 228 instructions, found {len(operations)}")
    require(depths(operations, inputs, outputs) == (35, 6), "unexpected ordinary depth or AND-depth")

    for x, table_value in enumerate(FIPS_SBOX):
        algebraic = aes_sbox_algebraic(x)
        require(algebraic == table_value,
            f"FIPS algebraic definition and literal table disagree at {x:02x}: {algebraic:02x} != {
                table_value:02x}")
        obtained = evaluate(operations, inputs, outputs, x)
        require(obtained == table_value,
            f"input {x:02x}: obtained {obtained:02x}, expected {table_value:02x}")

```

```

digest = hashlib.sha256(path.read_bytes()).hexdigest()
if path.resolve() == DEFAULT.resolve():
    require(digest == DEFAULT_SHA256, f"default certificate SHA-256 mismatch: {digest}")
print(
    "PASS: "
    f"{len(operations)} instructions; {counts['AND']} AND, {counts['XOR']} XOR, "
    f"{counts['NOT']} NOT; depth 35; AND-depth 6; all 256 FIPS entries; sha256 {digest}"
)

if __name__ == "__main__":
    main()

```

References

- [1] National Institute of Standards and Technology. *Circuit Complexity*. URL: <https://csrc.nist.gov/projects/circuit-complexity> (visited on 07/12/2026).
- [2] National Institute of Standards and Technology. *Circuit Complexity: List of Circuits*. URL: <https://csrc.nist.gov/projects/circuit-complexity/list-of-circuits> (visited on 07/12/2026).
- [3] National Institute of Standards and Technology. *aes-sbox-fwd-g113-a32-d27-ad6.slp*. *Official forward AES S-box straight-line program in the usnistgov/Circuits repository*. URL: <https://github.com/usnistgov/Circuits/blob/master/data/slp/aes/aes-sbox-fwd-g113-a32-d27-ad6.slp> (visited on 07/12/2026).
- [4] National Institute of Standards and Technology. *Advanced Encryption Standard (AES)*. Federal Information Processing Standards Publication 197-upd1. National Institute of Standards and Technology, May 9, 2023. URL: <https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.197-upd1.pdf> (visited on 07/12/2026).
- [5] Joan Boyar, Philip Matthews, and René Peralta. “Logic Minimization Techniques with Applications to Cryptology”. In: *Journal of Cryptology* 26.2 (2013), pp. 280–312.